

Golang 核心知识点清单（详细版）

面向有一定 C / C++ 基础的读者，按「基础 → 进阶」顺序系统梳理 Go 语言核心特性。
每个模块除概念讲解与代码示例外，均附 **与 C / C++ 的区别**，帮助建立对照。

目录

- [O、Go 与 C / C++ 总体对比](#)
 - [一、基础语法](#)
 - [二、函数与指针](#)
 - [三、核心数据结构](#)
 - [四、类型与面向对象抽象](#)
 - [五、并发编程基础](#)
 - [六、错误处理](#)
 - [七、高级特性与工程化](#)
 - [八、Go 设计哲学小结](#)
-

O、Go 与 C / C++ 总体对比

先建立全局印象，再逐点展开。

维度	C	C++	Go
设计年代	1972	1985 (C++11 起现代化)	2009
语言范式	过程式	过程式 + 面向对象 + 泛型	过程式 + 组合 (非继承) + 并发
内存管理	手动 <code>malloc/free</code>	手动 <code>new/delete</code> + 智能指针 (RAII)	垃圾回收 (GC), 无手动释放
类型系统	弱类型、隐式转换多	强类型、支持隐式转换	强类型、 几乎无隐式转换
空值表示	<code>NULL</code>	<code>nullptr</code>	<code>nil</code>
错误处理	返回值 / <code>errno</code>	异常 (<code>try/throw/catch</code>)	错误值 (<code>error</code>) + <code>panic/recover</code>
并发模型	线程 (pthread) + 锁	线程 (<code>std::thread</code>) + 锁/原子	Goroutine + Channel (CSP)
面向对象	无	类、继承、多态、虚函数	结构体 + 方法 + 接口 (隐式实现)
泛型	无 (宏模拟)	模板 (编译期)	类型参数 (Go 1.18+)
编译产物	原生二进制	原生二进制	原生二进制 (静态、自带运行时/GC)
头文件	有 <code>.h</code>	有 <code>.h/.hpp</code>	无头文件 , 按包组织
包管理	无标准方案	无官方方案 (vcpkg/Conan)	内置 <code>go mod</code>
测试	无内置	无内置 (GoogleTest 等)	内置 <code>testing</code> + <code>go test</code>
代码风格	自由	自由	<code>gofmt</code> 强制统一

一句话定位: C 控制一切、C++ 提供抽象与性能、Go 用「简单的语法 + 内置并发 + GC」换取开发效率与大规模并发工程能力。

一、基础语法

1.1 变量与常量声明

Go 的声明语法是**名称在前、类型在后**, 与 C/C++ 的「类型在前」相反。

```
// var 声明 (类型可省略, 由编译器推断)
var a int = 10
var b = 20           // 推断为 int
```

```

var c string          // 零值 ""

// 批量声明
var (
    x int    = 1
    y string = "hello"
)

// 短变量声明（仅函数内部可用，最常用）
d := 30
name, age := "colin", 20 // 可同时声明多个，自动推断

// 常量
const Pi = 3.14159
const (
    StatusOK    = 200
    StatusFail  = 500
)

// iota: 常量计数器，每行自动 +1，适合定义枚举
const (
    Monday = iota // 0
    Tuesday      // 1
    Wednesday    // 2
)

```

与 C / C++ 的区别

点	C / C++	Go
声明方向	<code>int a = 10;</code> (类型在前)	<code>var a int = 10</code> (类型在后)
短声明	无 (C++11 有 <code>auto a = 10;</code>)	<code>a := 10</code> (函数内), <code>auto</code> 仅推类型、 <code>:=</code> 同时声明+赋值
未初始化变量	未定义行为 / 垃圾值 (局部变量)	自动获得 零值 (数值 0、字符串 ""、bool false、指针 nil)
常量	<code>#define</code> 宏 / <code>const</code> (C++ 可 <code>constexpr</code>)	<code>const</code> , 且 <code>iota</code> 生成枚举; <code>const</code> 只能是基本类型的编译期常量
分号	每条语句以 <code>;</code> 结尾	语句末尾分号可省略 (词法器自动插入)
未使用变量	仅警告	编译错误 (未使用的变量/导入会直接编译失败)

1.2 基本数据类型

Go 内置类型一览:

分类	类型	说明
布尔	<code>bool</code>	取值 <code>true</code> / <code>false</code>
有符号整型	<code>int</code> <code>int8</code> <code>int16</code> <code>int32</code> <code>int64</code>	<code>int</code> 长度随平台 (32/64 位)
无符号整型	<code>uint</code> <code>uint8</code> <code>uint16</code> <code>uint32</code> <code>uint64</code>	另有 <code>uintptr</code> 存放指针地址
别名	<code>byte</code> = <code>uint8</code> , <code>rune</code> = <code>int32</code>	<code>rune</code> 表示一个 Unicode 码点
浮点	<code>float32</code> <code>float64</code>	无 <code>float</code> 简写
复数	<code>complex64</code> <code>complex128</code>	C/C++ 标准库才有 (C99 <code><complex.h></code>)
字符串	<code>string</code>	不可变的 UTF-8 字节序列

```

var b bool = true
var i int = 42
var u uint8 = 255
var f float64 = 3.14
var s string = "你好, Go" // 内置类型, 不是指针
var r rune = '中'         // 单引号是 rune (int32), 不是 byte

// 类型转换: 必须显式, 不能隐式
var x int = 10
var y float64 = float64(x) // 显式转换
// var y float64 = x       // 编译错误: 不能隐式转换

// 零值
var zeroInt int    // 0
var zeroStr string // ""
var zeroBool bool  // false
var zeroPtr *int   // nil

```

与 C / C++ 的区别

点	C / C++	Go
布尔	C99 才有 <code>_Bool / bool</code> ，且可与整型互转（ <code>if(1)</code> 成立）	<code>bool</code> 是独立类型， 不能与 int 互转
字符	<code>char</code> （1 字节）	<code>rune</code> （int32，存 Unicode 码点）； <code>byte</code> 才是 1 字节
字符串	C 是 <code>char[] / char*</code> （以 <code>\0</code> 结尾）；C++ 是 <code>std::string</code>	<code>string</code> 内置值类型，不可变、UTF-8、有长度信息
隐式转换	大量隐式算术转换（如 <code>int→double</code> ）	几乎不允许隐式转换 ，必须 <code>T(v)</code> 显式转换
整数长度	各平台/编译器不一（ <code>int</code> 通常 32 位）	<code>int</code> 明确随平台 32/64 位，其余 <code>int8~64</code> 长度固定
指针运算	支持 <code>p++</code> 、 <code>p+i</code> 等	普通指针 无指针运算 （需 <code>unsafe.Pointer</code> ）

1.3 控制结构

if / else

Go 的条件**不加括号**，但**大括号必须写**；支持在条件前加一个初始化语句。

```
if x := compute(); x > 0 { // 先声明 x，再判断；x 作用域仅在 if 内
    fmt.Println("positive")
} else if x < 0 {
    fmt.Println("negative")
} else {
    fmt.Println("zero")
}
```

for（Go 唯一的循环）

Go 没有 `while` 和 `do-while`，全部用 `for` 表达。

```
// 1. 经典三段式
for i := 0; i < 10; i++ { fmt.Println(i) }

// 2. 相当于 while
for x < 100 { x *= 2 }

// 3. 无限循环
for { /* ... */ }

// 4. for range 遍历
nums := []int{1, 2, 3}
for idx, val := range nums { fmt.Println(idx, val) }
for key, val := range m { /* 遍历 map */ }
```

switch

Go 的 switch **默认不贯穿** (无需 `break`) , 且 `case` 可以是表达式。

```
switch n {
case 1:
    fmt.Println("one")    // 自动 break, 不会落到下一个 case
case 2, 3:
    fmt.Println("two or three")
default:
    fmt.Println("other")
}

// 无表达式的 switch: 相当于 if-else 链
switch {
case score >= 90: fmt.Println("A")
case score >= 60: fmt.Println("B")
default:          fmt.Println("C")
}

// 需要贯穿时显式写 fallthrough
switch n {
case 0:
    fmt.Println("start")
    fallthrough
case 1:
    fmt.Println("one")
}
```

select (通道多路复用)

`select` 是 Go 并发独有的控制结构: 在多个通道操作中选择一个**就绪**的分支执行, 无 C/C++ 直接对应物。

```
select {
case msg := <-ch1:
    fmt.Println("from ch1:", msg)
case msg := <-ch2:
    fmt.Println("from ch2:", msg)
case ch3 <- 42:
    fmt.Println("sent to ch3")
default:
    fmt.Println("no channel ready")    // 可选: 非阻塞
}
```

与 C / C++ 的区别

点	C / C++	Go
if 括号	必须 <code>if (x)</code>	不写括号 <code>if x</code>
大括号	可省略（单语句）	必须写
循环	<code>for</code> / <code>while</code> / <code>do-while</code> 三种	仅 <code>for</code> 一种（可变出 <code>while</code> 形式）
switch 穿透	默认贯穿 ，需 <code>break</code> 阻断	默认不贯穿 ，需 <code>fallthrough</code> 才贯穿
switch 条件	仅常量/整数表达式	<code>case</code> 可为任意表达式、多个值、无表达式、类型
三目运算符	有 <code>a ? b : c</code>	没有 ，用 <code>if-else</code> 代替
<code>select</code>	无（C 的 <code>select()</code> 是 socket 多路复用，非此语义）	通道多路复用，Go 独有

二、函数与指针

2.1 函数定义与多返回值

```
// 基本定义: func 名称(参数) 返回值类型
func add(a int, b int) int { return a + b }
func add2(a, b int) int { return a + b } // 同类型参数可合并书写

// 多返回值 (Go 的核心惯例)
func divide(a, b int) (int, error) {
    if b == 0 { return 0, errors.New("division by zero") }
    return a / b, nil
}

// 命名返回值: 返回值有名字, 可直接 return
func split(sum int) (x, y int) {
    x = sum * 4 / 9
    y = sum - x
    return // 裸 return, 返回 x, y
}

// 可变参数
func sum(nums ...int) int {
    total := 0
    for _, n := range nums { total += n }
    return total
}
sum(1, 2, 3, 4)
```

与 C / C++ 的区别

点	C / C++	Go
定义语法	<code>int add(int a, int b){}</code>	<code>func add(a, b int) int {}</code> (类型在后)
多返回值	只能返回一个值，用指针/引用/结构体返回多个	原生支持多返回值 ，惯用「结果 + error」
可变参数	C <code>printf</code> 用 <code>... + va_list</code> ; C++ 用模板 <code>initializer_list</code>	<code>func f(args ...int)</code> ，参数以切片传入
参数传递	C 值传递; C++ 值传递 + 引用传递	只有值传递 （传指针/切片等也是传它们的值）
命名返回值	无	有，可简化 <code>return</code>

2.2 匿名函数与闭包

```
// 匿名函数：可赋值、可立即执行
f := func(a, b int) int { return a + b }
fmt.Println(f(1, 2))

// 立即执行
func() { fmt.Println("IIFE") }()

// 闭包：函数捕获外部变量
func counter() func() int {
    count := 0
    return func() int { // 返回的函数持有 count 的引用
        count++
        return count
    }
}
c := counter()
fmt.Println(c(), c(), c()) // 1 2 3
```

与 C / C++ 的区别

点	C / C++	Go
匿名函数	C 无; C++11 有 <code>lambda []()</code> <code>{}</code>	<code>func(...) {...}</code>
闭包捕获	C++ 需显式指定捕获列表 <code>[=]</code> / <code>[&]</code>	自动捕获 外层变量，无需声明
变量逃逸	手动管理生命周期	闭包引用的变量若逃逸，编译器自动分配到堆并由 GC 回收

2.3 defer 延迟调用

`defer` 把函数调用推迟到外层函数返回之前执行，常用于资源释放。

```
func readFile(path string) error {
    f, err := os.Open(path)
    if err != nil { return err }
    defer f.Close()           // 无论函数如何返回，close 都会执行

    // ... 处理文件 ...
    return nil
}

// 多个 defer: 按后进先出 (LIFO) 执行
func demo() {
    defer fmt.Println("A")    // 最后执行
    defer fmt.Println("B")    // 中间执行
    defer fmt.Println("C")    // 最先执行
    // 输出: C B A
}

// defer 参数在 defer 语句处立即求值
func demo2() {
    x := 1
    defer fmt.Println(x)      // 此时 x=1 已被捕获，最终打印 1
    x = 100
}
```

与 C / C++ 的区别

点	C / C++	Go
等价机制	C 无; C++ 用 RAII (析构函数) 自动释放	<code>defer</code> 显式注册，返回前统一执行
释放时机	作用域/对象析构时	函数返回前 (LIFO 顺序)
典型用途	智能指针、lock_guard、fstream	<code>close</code> 、 <code>unlock</code> 、 <code>recover</code> 搭配使用

Go 的 `defer` 与 C++ 的 RAII 目标相同 (确保资源释放)，但实现不同：RAII 靠类型析构，`defer` 靠语句注册。

2.4 指针

```
func main() {
    x := 10
    p := &x           // p 是指向 x 的指针，类型 *int
    fmt.Println(*p)    // 10: 解引用读取
    *p = 20            // 通过指针修改 x
    fmt.Println(x)     // 20

    q := new(int)      // new: 分配内存并返回指向零值的指针
}
```

```
fmt.Println(*q)    // 0

inc(&x)            // 传地址，函数内修改外部变量
fmt.Println(x)    // 21
}

// 指针参数 + 自增（注意：必须写 (*n)++，Go 中 *n++ 是语法错误，与 C 的 *p++ 不同）
func inc(n *int) { (*n)++ }
```

典型使用场景：

- 1. 函数需要修改外部变量时传指针；
- 2. 传递大结构体避免整份拷贝；
- 3. 表达「可为空」的引用语义（nil）；
- 4. 与切片/map 等共享底层数据配合。

与 C / C++ 的区别

点	C / C++	Go
取地址/解引用	& / *（语法一致）	& / * 相同
指针运算	支持 p++、p+n、数组指针运算	不支持（需 unsafe 包）
引用	C++ 有 T& 引用	没有引用类型，统一用指针
多级指针	常见 int**	少见，语义上尽量用值/切片代替
空指针	NULL / nullptr	nil
内存释放	free / delete	由 GC 自动回收，无手动释放
悬垂指针	需自行避免	GC 保证对象存活期，但切片/接口仍有逃逸问题 需注意

三、核心数据结构

3.1 数组与切片

数组（Array）：长度固定的值类型

```
var a [3]int      // [0 0 0]
b := [3]int{1, 2, 3}
c := [...]int{1, 2, 3} // 长度由编译器推断

// 数组是值类型：赋值/传参都会整体拷贝
d := b
d[0] = 999
fmt.Println(b[0])    // 仍为 1（b 未被修改）

// [3]int 与 [4]int 是不同的类型
```

切片 (Slice)：长度可变的动态视图

切片是 Go 最常用的数据结构，本质是一个指向底层数组的窗口，由三部分组成：指针 + 长度 len + 容量 cap。

```
// 创建方式
s1 := []int{1, 2, 3}           // 字面量（注意无长度）
s2 := make([]int, 3)           // len=3, cap=3, 元素零值
s3 := make([]int, 3, 10)       // len=3, cap=10

// 从数组/切片切出切片（共享底层数组）
arr := [5]int{1, 2, 3, 4, 5}
s4 := arr[1:4]                 // [2 3 4], len=3, cap=4
s4[0] = 99
fmt.Println(arr)               // [1 99 3 4 5]: 底层数组被修改

// append: 追加元素，容量不足时自动扩容并返回新切片
s := []int{1, 2}
s = append(s, 3, 4, 5)         // 必须接收返回值

// copy: 复制元素，长度为两者较小值
dst := make([]int, 3)
copy(dst, s)

// 遍历
for i, v := range s { fmt.Println(i, v) }
```

数组 vs 切片 对比

点	数组 [N]T	切片 []T
长度	固定，是类型的一部分	可变 (len)
语义	值类型，赋值即拷贝	引用语义，共享底层数组
存储	元素直接内嵌	指针 + len + cap
使用频率	较少（用于固定长度）	极高（Go 的主力集合类型）

与 C / C++ 的区别

点	C / C++	Go
数组传参	C 数组退化为指针（丢失长度）；C++ std::array 是值类型	数组是值类型，传参整体拷贝
动态数组	C++ std::vector（拥有内存、可扩容）	切片（视图，底层数组可能被共享）
扩容	vector.push_back 自动重分配	append，必要时自动扩容（一般 2 倍增长）
越界	C 不检查 (UB)；C++ operator[] 不检查、at() 检查	运行时自动 panic（索引越界即崩溃）
长度	C 需手动传；C++ 有 .size()	内置 len() / cap()

切片共享底层数组是一把双刃剑：高效但可能产生「别名」副作用；`append` 扩容后可能脱离原数组，需注意。

3.2 映射（Map）

`map` 是无序的键值对集合，键类型必须**可比较**（`==` 合法），是引用类型。

```
// 创建
m := map[string]int{"a": 1, "b": 2}
m2 := make(map[string]int) // 空 map（非 nil，可写）

var m3 map[string]int      // nil map，写入会 panic

// 增改
m["c"] = 3
m["a"] = 100                // 覆盖

// 删
delete(m, "b")

// 查（第二个返回值判断键是否存在）
v, ok := m["a"]
if ok { fmt.Println(v) } else { fmt.Println("not found") }

// 只判断存在性
if _, exists := m["x"]; exists { /* ... */ }

// 遍历（顺序不保证）
for k, v := range m { fmt.Println(k, v) }
```

与 C / C++ 的区别

点	C / C++	Go
内置映射	C 无（需手写哈希表）	内置 <code>map</code>
C++ 对应	<code>std::map</code> （有序红黑树）/ <code>std::unordered_map</code> （哈希）	类似 <code>unordered_map</code> ， 无序 、哈希实现
查询存在性	<code>find() != end()</code> ，或 <code>operator[]</code> （不存在会插入默认值）	<code>v, ok := m[k]</code> 用第二个返回值判断
遍历顺序	<code>std::map</code> 有序； <code>unordered_map</code> 无序	无序（且刻意随机化）
线程安全	需自行加锁	非线程安全 ，并发读写需 <code>sync.Mutex</code> 或 <code>sync.Map</code>

四、类型与面向对象抽象

Go **没有 class 和继承**，用「结构体 + 方法 + 接口」实现面向对象抽象。

4.1 结构体与方法

```
// 定义结构体
type Person struct {
    Name string
    Age  int
}

// 字段标签（供反射/序列化使用，C/C++ 无对应物）
type User struct {
    ID    int    `json:"id"`
    Name string `json:"name"`
}

// 方法：在 func 与函数名之间加「接收者」
func (p Person) Greet() string {           // 值接收者：只读，不修改 p
    return "Hi, " + p.Name
}

func (p *Person) Birthday() {              // 指针接收者：可修改 p，避免拷贝
    p.Age++
}

// 使用
p := Person{Name: "colin", Age: 20}
p.Greet()
p.Birthday()           // 自动取地址，等价 (&p).Birthday()
```

值接收者 vs 指针接收者：

接收者	能否修改原值	拷贝开销	适用场景
值 (p T)	否（操作副本）	有（结构体较大时明显）	只读、小类型、值语义
指针 (p *T)	是	无（仅传指针）	需修改、大结构体、保持一致

与 C / C++ 的区别

点	C / C++	Go
类	C++ class/struct 封装数据+方法	数据用 struct，方法定义在外部（带接收者）
继承	C++ 支持单/多继承	无继承，用组合/嵌入（embedding）复用
this 指针	隐式 this	显式接收者（p *Person），命名自定义
构造/析构	构造函数、析构函数	无，惯例用 NewXxx() 工厂函数；defer 做清理
访问控制	public/private/protected	靠首字母大小写：大写=导出，小写=包内私有
运算符重载	支持	不支持

4.2 接口 (Interface)

接口声明一组**方法签名**，描述「能做什么」。Go 的接口是**隐式实现**——类型只要实现了接口的全部方法，就自动满足该接口，无需显式声明。

```
// 定义接口
type Writer interface {
    Write([]byte) (int, error)
}

// 任意实现了 Write 方法的类型都自动是 Writer
type MyWriter struct{}
func (m MyWriter) Write(p []byte) (int, error) { return len(p), nil }

var w Writer = MyWriter{} // 隐式实现，无需 implements 关键字

// 空接口：可持有任意类型的值
var any interface{} = 42
any = "hello"
any = struct{}{}

// 多态：同一接口可指向不同实现
type Shape interface{ Area() float64 }
type Circle struct{ R float64 }
func (c Circle) Area() float64 { return 3.14 * c.R * c.R }
type Rect struct{ W, H float64 }
func (r Rect) Area() float64 { return r.W * r.H }

func totalArea(shapes []Shape) float64 {
    sum := 0.0
    for _, s := range shapes { sum += s.Area() }
    return sum
}
```

接口值内部结构：接口值由两部分组成——**动态类型 + 动态值**（type, value 对），这就是它能实现多态和运行时断言的根基。

与 C / C++ 的区别

点	C / C++	Go
抽象机制	C++ 抽象基类 + 虚函数 (virtual)	接口 (方法集合)
实现关系	显式继承 class A : public Base	隐式实现 (结构化类型/鸭子类型)
空接口对应	C void*; C++17 std::any	interface{} / any
多态	虚函数表 (vtable) 动态分派	接口的动态分派
组合	多重继承	接口可多实现、类型可多接口

隐式实现是 Go 接口最鲜明的设计：**解耦**了「定义」与「实现」，新增实现无需改动接口定义方。

4.3 类型断言 (Type Assertion)

类型断言用于把接口值还原为具体类型。

```
var i interface{} = "hello"

// 1. 直接断言：类型不符合会 panic
s := i.(string)
fmt.Println(s)

// 2. 安全断言（推荐）：用两个返回值，不 panic
s, ok := i.(string)
if ok { fmt.Println(s) } else { fmt.Println("not a string") }

// 3. type switch: 按动态类型分情况处理
func describe(v interface{}) {
    switch v.(type) {
    case int:    fmt.Println("int")
    case string: fmt.Println("string")
    case bool:   fmt.Println("bool")
    default:    fmt.Println("unknown")
    }
}
```

与 C / C++ 的区别

点	C / C++	Go
类型判断	C++ <code>dynamic_cast</code> (需多态/虚函数)、 <code>typeid</code>	<code>x.(T)</code> / <code>type switch</code>
失败行为	<code>dynamic_cast</code> 指针失败返回 <code>nullptr</code> , 引用失败抛异常	直接断言失败 panic ; 安全断言返回 <code>ok=false</code>
通用容器解包	C++17 <code>std::any_cast</code> (失败抛异常)	<code>x.(T)</code> 类似
C 对应	无 (无 RTTI)	—

五、并发编程基础

Go 的核心竞争力是**内置并发**：Goroutine (协程) + Channel (通道)，遵循 CSP「通过通信共享内存」的哲学。

5.1 协程 (Goroutine)

```
// 用 go 关键字启动一个并发执行的函数
func printMsg(s string) {
    for i := 0; i < 3; i++ { fmt.Println(s, i) }
}

func main() {
    go printMsg("goroutine")    // 异步执行
    printMsg("main")           // 主协程同步执行
    // 两处输出会交错
}

// 匿名函数 + goroutine
go func(x int) { fmt.Println(x) }(42)
```

特点:

- **轻量**: 初始栈约 2KB, 可动态增长; 可轻松创建成千上万个;
- **由 Go 运行时调度**: 不是 1:1 操作系统线程, 而是多路复用到少量 OS 线程 (M:N 调度);
- **并发 ≠ 并行**: 单核上 goroutine 靠调度器切换实现并发, 多核上才真正并行。

与 C / C++ 的区别

点	C / C++	Go
并发单元	线程: <code>pthread_create</code> / <code>std::thread</code>	goroutine (轻量协程)
开销	每个线程约 1MB 栈 + 内核调度	约 2KB 起, 运行时调度
数量级	成百上千已吃力	可轻松 十万级
调度	操作系统内核调度	Go 运行时 M:N 调度
数据共享	共享内存 + 锁	推荐用 Channel 通信 (也可用锁)

5.2 通道 (Channel)

Channel 是 goroutine 之间**通信的管道**, 是 Go 并发的核心。

```
// 创建
ch := make(chan int)           // 无缓冲通道
ch2 := make(chan int, 10)      // 有缓冲通道 (容量 10)

// 发送 / 接收 (用 <- 箭头表示方向)
ch <- 42                        // 发送
v := <-ch                      // 接收

// 无缓冲通道: 发送与接收必须同步配对
func main() {
    ch := make(chan string)
    go func() { ch <- "ping" }() // 发送会阻塞, 直到有人接收
    msg := <-ch                 // 接收解除发送的阻塞
    fmt.Println(msg)
}

// 有缓冲通道: 缓冲区未满时发送不阻塞, 非空时接收不阻塞
```

```

ch := make(chan int, 2)
ch <- 1      // 不阻塞
ch <- 2      // 不阻塞
// ch <- 3    // 阻塞（缓冲区满）
fmt.Println(<-ch, <-ch)

// 关闭通道
close(ch)
// 接收时第二个返回值判断通道是否已关闭
v, ok := <-ch
if !ok { fmt.Println("channel closed") }

// for range 遍历直到通道关闭
for v := range ch { fmt.Println(v) }

// 只读/只写方向约束（增强类型安全）
func sendOnly(ch chan<- int) { ch <- 1 } // 只能发送
func recvOnly(ch <-chan int) { <-ch }   // 只能接收

```

无缓冲 vs 有缓冲：

通道	行为	用途
无缓冲 <code>make(chan T)</code>	发送/接收 必须同步 ，双方同时就绪才完成	同步、握手、事件通知
有缓冲 <code>make(chan T, n)</code>	可暂存 n 个元素，未满/非空时各自不阻塞	解耦、限流、生产者-消费者

与 C / C++ 的区别

点	C / C++	Go
通信原语	无内置通道，用共享内存+锁/信号量/管道	内置 Channel，一等公民
编程范式	「通过共享内存通信」	「 通过通信共享内存 」（CSP）
select	C 的 <code>select()</code> 是 socket 多路复用	<code>select</code> 是通道多路复用
同步原语对应	条件变量、信号量模拟	无缓冲通道天然提供同步

5.3 基本并发控制

sync.WaitGroup（等待一组 goroutine 完成）

```

var wg sync.WaitGroup

for i := 0; i < 5; i++ {
    wg.Add(1)           // 计数 +1
    go func(n int) {
        defer wg.Done() // 完成时计数 -1
        fmt.Println(n)
    }(i)
}
wg.Wait()              // 阻塞直到计数为 0
fmt.Println("all done")

```

sync.Mutex (互斥锁, 保护共享变量)

```

var (
    counter int
    mu      sync.Mutex
)

func increment() {
    mu.Lock()    // 加锁
    counter++    // 临界区
    mu.Unlock()  // 解锁
}

```

数据竞争 (Data Race) : 多个 goroutine 并发读写同一变量且至少有一个是写、且未同步时, 结果是不确定的。可用 `go run -race` 检测。

```

// 错误示范: 并发自增无锁, 产生数据竞争
var n int
for i := 0; i < 1000; i++ {
    go func() { n++ }() // race! 结果不确定
}

```

与 C / C++ 的区别

点	C / C++	Go
互斥锁	<code>std::mutex</code> / <code>pthread_mutex</code>	<code>sync.Mutex</code>
等待组	<code>std::latch/barrier</code> (C++20) 或手动计数	<code>sync.WaitGroup</code>
竞态检测	ThreadSanitizer (编译选项)	内置 <code>go run -race</code>
原子操作	<code>std::atomic</code>	<code>sync/atomic</code>
锁的释放	RAII (<code>lock_guard/unique_lock</code>) 自动	手动 <code>unlock</code> (常配合 <code>defer mu.Unlock()</code>)

六、错误处理

Go **没有异常机制**，错误以「值」的形式通过返回值传递，这是与 C/C++ 最大的分歧点之一。

6.1 Error 接口

`error` 是内置接口，只要求实现一个方法：

```
type error interface {
    Error() string
}
```

```
// 创建错误
import "errors"
err1 := errors.New("something wrong")
err2 := fmt.Errorf("open %s failed: %w", filename, err1) // %w 包装底层错误

// 返回错误（惯例：最后一个返回值是 error）
func doSomething() (int, error) {
    if fail { return 0, errors.New("failed") }
    return 42, nil
}

// 调用者显式检查
v, err := doSomething()
if err != nil {
    // 处理错误
    log.Fatal(err)
}

// 错误包装与解包判断
if errors.Is(err, err1) { /* 判断是否是该错误（含包装链） */ }
var target *MyError
if errors.As(err, &target) { /* 判断并提取具体错误类型 */ }
```

与 C / C++ 的区别

点	C / C++	Go
错误传递	C: 返回码 / <code>errno</code> ; C++: 异常	错误值 + 返回值, 无异常
错误检查	C++ 靠 try/catch 集中处理	<code>if err != nil</code> 逐层显式处理
调用栈	异常自动展开栈	error 值不携带栈信息（可自行包装）
错误类型	异常类体系	<code>error</code> 接口 + 具体实现类型
错误链	异常可嵌套	<code>%w</code> + <code>errors.Is/As</code> 形成包装链

6.2 Panic 与 Recover

`panic` 用于**不可恢复的严重错误**，`recover` 仅在 `defer` 中可捕获。

```
// panic: 程序无法继续时触发
func mustPositive(x int) {
    if x < 0 {
```

```

        panic("x must be non-negative") // 触发 panic, 向上传播
    }
}

// recover: 在 defer 中捕获 panic, 使程序恢复
func safe(f func()) (err error) {
    defer func() {
        if r := recover(); r != nil { // 捕获 panic
            err = fmt.Errorf("recovered: %v", r)
        }
    }()
    f()
    return nil
}

// 使用原则
// - panic 用于程序 bug、不可恢复的致命错误（如数组越界、空指针解引用）
// - 常规业务错误一律返回 error, 不 panic
// - recover 只应在边界处（如 HTTP handler）兜底, 避免程序整体崩溃

```

与 C / C++ 的区别

点	C / C++	Go
异常机制	C++ <code>throw/try/catch</code>	<code>panic/recover</code> (不是异常 , 语义更受限)
使用频率	异常可用于常规错误流	panic 仅用于致命错误, 常规用 error
捕获方式	catch 任意位置	只能在 defer 中 recover
栈展开	有	有 (defer 逆序执行后继续上抛)

七、高级特性与工程化

7.1 泛型 (Generics, Go 1.18+)

Go 的泛型通过**类型参数**让函数与类型复用, 比 C++ 模板简单得多 (无模板元编程)。

```

// 泛型函数: 类型参数写在方括号中
func Max[T int | float64](a, b T) T { // T 受约束 int|float64
    if a > b { return a }
    return b
}

fmt.Println(Max(3, 5)) // 5
fmt.Println(Max(3.14, 2.71)) // 3.14

// 泛型类型
type Stack[T any] struct { // any 表示任意类型
    items []T
}

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }

// 自定义约束 (interface 定义类型集合)
type Number interface {
    int | int64 | float64
}

```

```
}
func Sum[T Number](nums []T) T {
    var total T
    for _, n := range nums { total += n }
    return total
}
```

与 C / C++ 的区别

点	C / C++	Go
泛型机制	C 无 (宏 / C11 <code>_Generic</code>) ; C++ 模板	类型参数 + 约束
展开方式	C++ 模板 编译期实例化 (可元编程、图灵完备)	编译期实例化, 但 能力受限 、语法简单
约束表达	C++20 <code>concept</code>	接口 + 类型集合 (<code>int float64</code>)
复杂度	高 (模板元编程、SFINAE)	低, 刻意保持简单

7.2 反射 (Reflection)

反射在**运行时**检查与操作类型和值，用于编写通用框架（序列化、ORM、依赖注入等）。

```
import "reflect"

func inspect(v interface{}) {
    t := reflect.TypeOf(v)      // 获取动态类型
    val := reflect.ValueOf(v)   // 获取动态值

    fmt.Println("type:", t)
    fmt.Println("kind:", t.Kind()) // 底层种类: struct/int/ptr...

    // 遍历结构体字段 (结合结构体标签)
    if t.Kind() == reflect.Struct {
        for i := 0; i < t.NumField(); i++ {
            field := t.Field(i)
            fmt.Println(field.Name, field.Tag.Get("json"))
        }
    }
}

// 通过反射修改值 (需传指针)
func setInt(v interface{}, n int) {
    rv := reflect.ValueOf(v)
    if rv.Kind() == reflect.Ptr && rv.Elem().CanSet() {
        rv.Elem().SetInt(int64(n))
    }
}
```

注意：反射性能较低、可读性差、易破坏类型安全，应优先用泛型/接口，仅在确需处理未知类型时使用。

与 C / C++ 的区别

点	C / C++	Go
运行时类型信息	C 无; C++ RTTI 仅 <code>typeid / dynamic_cast</code> (能力有限)	<code>reflect</code> 包能力完整
反射操作	C++ 几乎无法动态读写成员	可遍历字段、调用方法、修改值
结构体标签	无	<code>reflect</code> + 标签实现 <code>json</code> 等序列化
代价	—	运行时开销大, 编译期类型安全被削弱

7.3 Go Modules (包管理)

```
# 初始化模块, 生成 go.mod
go mod init example.com/myproject

# 添加依赖 (写入 go.mod / go.sum)
go get github.com/gin-gonic/gin

# 清理未使用的、补全缺失的依赖
go mod tidy

# 查看依赖
go list -m all
```

`go.mod` 声明模块路径与依赖版本:

```
module example.com/myproject

go 1.22

require (
    github.com/gin-gonic/gin v1.10.0
)
```

- `go.sum`: 记录每个依赖的校验和, 保证构建**可复现、防篡改**;
- 模块以版本号 (语义化版本) 管理, 支持私有仓库与 `replace` 替换。

与 C / C++ 的区别

点	C / C++	Go
官方包管理	无 (C++ 社区用 <code>vcpkg/Conan/CMake</code>)	内置 <code>go mod</code>
版本管理	各方案自定	语义化版本 + 最小版本选择
可复现构建	较难保证	<code>go.mod</code> + <code>go.sum</code> 锁定
依赖拉取	各工具自定	直接拉取版本控制仓库

7.4 常用标准库

库	作用	示例
<code>fmt</code>	格式化输入输出	<code>fmt.Printf</code> 、 <code>fmt.Sprintf</code> 、 <code>fmt.Scanf</code>
<code>io</code>	I/O 基础接口	<code>io.Reader</code> / <code>io.Writer</code> (流式处理基石)
<code>net/http</code>	HTTP 客户端与服务端	<code>http.Get</code> 、 <code>http.HandleFunc</code>
<code>encoding/json</code>	JSON 序列化/反序列化	<code>json.Marshal</code> / <code>json.Unmarshal</code>

```
// fmt
fmt.Printf("%s is %d years old\n", name, age)
s := fmt.Sprintf("value=%d", 42)

// io: 一切 I/O 抽象为 Reader/Writer
var r io.Reader = strings.NewReader("hello")

// net/http: 服务端
http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello, %s", r.URL.Path)
})
http.ListenAndServe(":8080", nil)

// encoding/json
type User struct {
    Name string `json:"name"`
    Age  int    `json:"age"`
}
b, _ := json.Marshal(User{"colin", 20}) // {"name":"colin","age":20}
var u User
json.Unmarshal(b, &u)
```

与 C / C++ 的区别

点	C / C++	Go
标准库规模	C 很小; C++ STL 偏数据结构/算法	丰富, 且覆盖网络/HTTP/JSON/加密等工程领域
HTTP 支持	需第三方库 (libcurl 等)	内置 <code>net/http</code> , 开箱即用
JSON	第三方 (如 nlohmann/json)	内置 <code>encoding/json</code> , 配合结构体标签
命名风格	C++ <code>std::string</code> (类)	Go 以包+函数为主, 接口用 <code>Reader/Writer</code>

7.5 编写测试 (testing)

Go 内置测试框架，测试文件以 `_test.go` 结尾，函数以 `Test` 开头。

```
// 文件: math_test.go
package main

import "testing"

// 单元测试
func TestAdd(t *testing.T) {
    got := add(1, 2)
    want := 3
    if got != want {
        t.Errorf("add(1,2) = %d; want %d", got, want)
    }
}

// 表驱动测试 (Go 社区推荐)
func TestDivide(t *testing.T) {
    tests := []struct {
        name      string
        a, b      int
        want      int
        wantErr   bool
    }{
        {"正常", 10, 2, 5, false},
        {"除零", 10, 0, 0, true},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got, err := divide(tt.a, tt.b)
            if (err != nil) != tt.wantErr {
                t.Fatalf("err = %v, wantErr %v", err, tt.wantErr)
            }
            if got != tt.want { t.Errorf("got %d, want %d", got, tt.want) }
        })
    }
}

// 基准测试
func BenchmarkAdd(b *testing.B) {
    for i := 0; i < b.N; i++ { add(1, 2) }
}
```

```
go test          # 运行测试
go test -v       # 详细输出
go test -cover   # 覆盖率
go test -race    # 竞态检测
go test -bench . # 基准测试
```

与 C / C++ 的区别

点	C / C++	Go
测试框架	无内置，用 GoogleTest / Catch2 等	内置 <code>testing</code>
测试命令	各框架自定	统一 <code>go test</code>
表驱动测试	手动组织	社区标准写法，配合 <code>t.Run</code>
基准测试	第三方（如 Google Benchmark）	内置 <code>BenchmarkXXX</code>
覆盖率/竞态	外部工具	<code>-cover</code> 、 <code>-race</code> 内置

八、Go 设计哲学小结

贯穿全篇的几条 Go 核心思想，也解释了它与 C/C++ 的差异根源：

1. **简单优于聪明**：语法极小、无继承/运算符重载/异常，宁可啰嗦也不引入复杂特性；
2. **显式优于隐式**：错误显式返回、类型显式转换、接口隐式但实现显式（靠方法集）；
3. **组合优于继承**：用结构体嵌入与接口组合代替类继承；
4. **并发是一等公民**：`go` + `channel`，用通信共享内存；
5. **工程化内置**：`gofmt`、`go test`、`go mod`、`go vet` 开箱即用；
6. **内存安全 + GC**：保留指针的表达力，去掉指针运算与手动释放；
7. **零值可用**：变量声明即有确定值，减少未初始化错误。

记忆对照：**C 让你掌控一切（性能/内存），C++ 给你抽象与泛型（复杂度高），Go 给你简单、并发与工程效率（牺牲部分极致性能与灵活性）。**

本清单按「基础语法 → 函数与指针 → 核心数据结构 → 类型与抽象 → 并发 → 错误处理 → 高级特性与工程化」组织，每节均可独立阅读。